android-avb-rollback

Android Verified Boot (AVB) / dm-verity / `boot_control` / Rollback — Stack Research Note

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Initial research complete. The on-device integrity-and-rollback chain that backs Android A/B updates is documented end-to-end from primary AOSP sources: the AVB chain of trust and `verifiedbootstate` colours; dm-verity hashtree + error modes; the `boot_control` HAL slot state machine (`setActiveBootSlot`, `markBootSuccessful`, `setSlotAsUnbootable`, retry-count fallback); `update_verifier` care-map verification and the path to `markBootSuccessful`; and AVB rollback-index / anti-downgrade with the load-bearing ordering rule (mark slot SUCCESSFUL **before** bumping stored rollback index). Conclusion: this stack delivers the “zero-corruption” guarantee on its own; the Helix agent must drive it through documented APIs and must NOT touch slot flags, rollback indexes, vbmeta, or verity metadata directly.
Issues	Some exact constants are version/board dependent and are tagged UNVERIFIED: the default `slot_retry_count` value (commonly 3/7), whether the RK3588 / Orange Pi 5 Max

Field	Value
	target build actually ships a conformant `boot_control` HAL + locked AVB, and the precise per-device rollback-index storage backend (RPMB vs persistent partition vs TEE). The exact `IBootControl` AIDL surface on Android 15 was not byte-confirmed against the tag and is flagged.
Fixed	N/A (first revision)
Continuation	Follow-up note should: (1) byte-confirm the Android 15 `android.hardware.boot` AIDL (`IBootControl`) method set and the `update_engine`? HAL call order; (2) confirm RK3588/Orange Pi 5 Max AVB lock state, vbmeta signing, and rollback-index storage; (3) trace the exact `update_verifier` → `setBootSuccessful`/ `markBootSuccessful` call site in current `bootable/recovery`; (4) document Helix telemetry hooks for verity EIO events and slot-fallback detection (`ro.boot.verifiedbootstate`, `ro.boot.veritymode`, current/active slot, `slot-successful`).

Table of Contents

1. Scope & Question

Helix OTA targets **Android 15 first**, using **native A/B + Virtual A/B** on device (see the companion note `aosp-update-engine.md`) with a **custom Go control plane** as the server. The companion note established that `update_engine` does the atomic write + slot switch. This note drills into the layer underneath that makes the update *safe*:

- What is the **Android Verified Boot (AVB)** chain of trust, and what does each `verifiedbootstate` colour mean?
- How does **dm-verity** guarantee the integrity of read-only partitions at runtime, and what happens on corruption?
- How does the **`boot_control` HAL** manage slot state (active / bootable / successful / unbootable / retry count)?
- How does **`update_verifier`** prove the freshly-written slot and reach `markBootSuccessful`?
- How does the bootloader perform **automatic rollback** on a failed boot?
- How do **rollback-index / anti-downgrade** protections work, and what is the ordering rule that keeps them from bricking a device?
- Together, how does this stack deliver the **zero-corruption guarantee**, and what must the **Helix agent do / not do** to preserve it?

The load-bearing claim to validate: *the safety guarantee is owned by AOSP + the bootloader, not by Helix; Helix must drive it through documented APIs and never manipulate the underlying flags/metadata.*

---

2. Executive Summary

- **AVB establishes a chain of trust** rooted in the bootloader. The bootloader verifies `vbmeta` (signed top-level metadata), which transitively authenticates every protected partition's descriptors. The result is reported to the OS via the kernel cmdline `androidboot.verifiedbootstate` with four states — **GREEN, YELLOW, ORANGE, RED** — and `androidboot.veritymode`. (AOSP Boot Flow.)
- **dm-verity** provides *transparent, per-block, on-read* integrity checking of read-only partitions using a **SHA-256 hash tree** whose single **root hash** is the only value that must be trusted (it is carried/authenticated by AVB). On a hash mismatch the kernel returns an **I/O error (EIO)** — "It appears as if the filesystem has been corrupted, as is expected." Android 7.0+ adds **Forward Error Correction (FEC)** (Reed–Solomon) to tolerate isolated bit-rot. (AOSP dm-verity.)
- **`boot_control` HAL** (`hardware/libhardware/include/hardware/boot_control.h`; modern AIDL `android.hardware.boot`) is the bootloader-implemented contract for slot management: **`getCurrentSlot`**, **`setActiveBootSlot`** (clears unbootable + successful markers, **resets the retry count**), **`markBootSuccessful`** (called by the **Android framework, never by the bootloader**), **`setSlotAsUnbootable`**, **`isSlotBootable`**. (AOSP Implement OTA updates.)
- **`update_verifier`** runs on the **first boot into a newly-written slot**. It reads the **care map** (the set of blocks actually written) and forces the kernel to read those blocks so **dm-verity verifies them** before the slot is committed. Only if

verification passes does the path to **`markBootSuccessful`** proceed. (`bootable/recovery/update_verifier`.)

- **Automatic rollback**: a freshly-activated slot starts **not-successful** with a positive **`slot-retry-count`**. Each boot attempt decrements the count; if the slot is never marked SUCCESSFUL and the count hits zero, the bootloader marks it unbootable and **falls back to the other slot** (which still holds the prior, known-good Android). This is the core safety property and it is **bootloader-enforced, not OS-enforced.**
- **Anti-rollback** prevents *downgrade* attacks: each image carries a **rollback index**; the device keeps a **stored rollback index** in tamper-evident storage; boot is permitted only if `image.rollback_index >= stored_rollback_index`. **Critical ordering rule:** the slot must be marked **SUCCESSFUL first**, and only *then* is the stored rollback index bumped — otherwise a power loss could leave the device unable to fall back to the previous (now disallowed) version.
- **Zero-corruption guarantee** = (atomic A/B write to the *inactive* slot) × (AVB-authenticated vbmeta) × (dm-verity per-block verify, with `update_verifier` forcing it on first boot) × (retry-count + automatic slot fallback) × (monotonic rollback index). Every one of these is on-device, signed, and battle-tested.
- **Helix's job** is to deliver a correctly-signed payload and *drive the documented update path*; **Helix must NOT** flip slot flags out of band, write rollback indexes, regenerate/strip vbmeta, disable verity, or call `markBootSuccessful` itself. Doing any of those would silently void the guarantee. Helix's legitimate value-add is **rollout/telemetry/recall orchestration** plus a correctly-built, correctly-signed OTA package.

---

3. The Verified Boot Chain of Trust

Source: Boot flow, Android Verified Boot (AVB), AVB 2.0 README.

Root of trust. AVB is the recommended default implementation for verifying the integrity of read-only partitions. It ships **`libavb`**, a C library used at boot time. The bootloader holds the OEM root key (and, on locked devices, optionally a user-settable root of trust). The bootloader verifies the **`vbmeta`** image — a signed, top-level structure that contains descriptors (hashtree descriptors, hash descriptors, **chain partition descriptors**, and rollback-index data) for the partitions it protects.

Chained partitions / delegation. A **chain partition descriptor** delegates authority: it names the partition and the public key authorized to sign that partition's own vbmeta footer. This is how `boot`, `vendor`, `system`, etc. can be signed/updated semi-independently while still rooting back to the top-level vbmeta the bootloader trusts. This is exactly the "delegating updates for different partitions" feature AVB advertises.

Verified boot states (reported via `androidboot.verifiedbootstate`):

- **GREEN** — "if the device is `LOCKED` and user-settable root of trust isn't used." Full chain of trust to the OEM key; normal secure boot.
- **YELLOW** — "if the device is `LOCKED` and user-settable root of trust is used." Boot is verified against a user-provided key (e.g., custom OS signed by a known key).

- **ORANGE** — "if the device is `UNLOCKED`." Verification is not enforced; the user has taken responsibility. A production Helix fleet should be **GREEN** (or YELLOW for a custom-key fleet).
- **RED** — dm-verity corruption (the device is `eio`/no valid OS) or no valid OS found. Boot is halted/last-resort.

The bootloader passes both `androidboot.verifiedbootstate` and `androidboot.veritymode` on the kernel command line, which is how userspace (and therefore the Helix agent) can *observe* — but not change — the trust state. For A/B devices the docs are explicit: **the boot slot must be marked `SUCCESSFUL` (via the Boot Control HAL) before rollback-protection metadata is updated** (see §8).

---

4. dm-verity: Transparent Integrity of Read-Only Partitions

Source: Implement dm-verity.

What it is. A device-mapper kernel target (Android 4.4+) that provides *transparent integrity checking* of a block device. Its purpose is to prevent **persistent rootkits** and ensure the device boots an uncompromised read-only image.

Hash tree. Every 4 KiB block of the protected partition is SHA-256 hashed; those hashes are concatenated and hashed again, layer by layer, up to a single **root hash**. Only the root hash must be trusted — and that trust is supplied by **AVB** (the root hash lives in / is authenticated by the signed vbmeta/verity metadata). Any modification to any block breaks the chain up to the root.

Verity metadata block. Bundled into a ~32 KiB block containing: magic number `0xb001b001`, a version field, an **RSA-2048 signature**, the table length, and the dm-verity table. A public key (historically on the boot partition; under AVB, anchored through vbmeta) validates the signature.

On-read verification. Blocks are verified as they are read into memory (hashing happens in parallel with the already-expensive disk read, so latency is effectively hidden). Verification is *lazy and continuous*, not a one-shot scan — which is why `update_verifier` is needed to *force* verification of freshly-written blocks before commit (§6).

Forward Error Correction (FEC). Android 7.0+ adds Reed–Solomon error-correcting codes with interleaving, letting dm-verity *recover* from isolated corruption (bit-rot, marginal flash) instead of failing the read.

Error handling. "If verification fails, the device generates an **I/O error** indicating the block can't be read. It appears as if the filesystem has been corrupted, as is expected." Apps may continue if the unreadable data is non-critical but fail when required data is inaccessible. The behaviour on verity failure is governed by the **verity mode** (e.g., enforcing/restart vs **EIO**), surfaced via `ro.boot.veritymode` — this distinction matters for how `update_verifier` treats a first-boot failure (§6).

---

5. `boot_control` HAL: The Slot State Machine

Source: Implement OTA updates; HAL interface `hardware/libhardware/include/hardware/boot_control.h` (modern AIDL: `android.hardware.boot` / `IBootControl`).

The `boot_control` HAL is the **bootloader-implemented** contract that lets userspace (`update_engine`, `update_verifier`, the framework) read and mutate slot metadata. The bootloader must implement it for A/B to work. Key operations (names from the AOSP page; AIDL spellings differ slightly):

- **`getCurrentSlot()`** — which slot the bootloader loaded / will attempt to load.
- **`setActiveBootSlot(slot)`** — make a slot active for next boot. Per the HAL semantics: it **updates the current slot, clears the unbootable and successful markers, and resets the retry count** to its positive starting value. This is what `update_engine` calls after writing the inactive slot.
- **`markBootSuccessful()`** — mark the currently-running slot as successfully booted. **Called by the Android framework, never by the bootloader.** This is the commit that ends the trial period.
- **`setSlotAsUnbootable(slot)`** — mark a slot unusable (e.g., a failed/incomplete write).
- **`isSlotBootable(slot)`** — query bootability.

Per-slot metadata the bootloader maintains: **active**, **bootable**, **successful**, and a **`slot-retry-count`**. The interplay of these flags + the retry count is the actual rollback mechanism (§7). The retry count can also be reset to a positive value (usually 3 — **UNVERIFIED** exact default; board-dependent) by `setActiveBootSlot` or by the fastboot `set_active` command.

---

6. `update_verifier`: Proving the New Slot Before Commit

Source: `update_verifier.cpp` (in `platform/bootable/recovery`), AOSP A/B docs.

After `update_engine` writes the inactive slot and `setActiveBootSlot` switches to it, the device reboots into the new slot in a **trial state** (bootable, not-yet-successful, retry-count > 0). Before that slot can be committed, the freshly-written blocks must actually pass dm-verity. That is `update_verifier`'s job:

1. It runs early on first boot into the new slot (before `zygote`/full system start, per the A/B docs).
2. It reads the **care map** (`care_map.txt` / `care_map.pb`) — the set of blocks that were actually written by the OTA — so it only verifies *relevant* blocks rather than the whole partition.
3. It **forces reads** of those blocks, which causes the kernel **dm-verity** target to verify them against the hash tree. It checks the **dm-verity mode** (`ro.boot.veritymode`) and handles **EIO mode** differently from **enforcing mode** (in EIO mode a corrupt block yields an I/O error the verifier can observe rather than an immediate restart).

4. If all care-map blocks verify, `update_verifier` reports success, and the path to **`markBootSuccessful()`** (via the `boot_control` HAL) proceeds — committing the slot. If verification fails, the slot is **not** marked successful, leading to fallback (§7).

The net effect: dm-verity's normally-lazy verification is *forced eagerly* on exactly the blocks the OTA touched, so a corrupt download/write is caught **before** the device gives up its known-good fallback slot.

---

7. Automatic Rollback on Failed Boot

Source: Implement OTA updates, Boot flow, AOSP A/B docs.

The rollback safety net is **bootloader-enforced** and works purely off the slot metadata:

1. After an update, the new slot is **active, bootable, NOT successful**, with `slot-retry-count` reset to a positive value.
2. On each boot attempt of a not-yet-successful slot, the bootloader **decrements `slot-retry-count`**.
3. If the OS boots far enough to verify (`update_verifier` passes) and the framework calls **`markBootSuccessful`**, the slot becomes **successful** and the retry mechanism stops — the update is committed.
4. If the slot is never marked successful and **`slot-retry-count` reaches zero**, the bootloader marks the slot unusable and **falls back to the other slot marked `slot-successful`** — which still contains the previous, known-good Android. Quoting the AOSP behaviour: "If there's a platform update that fails (isn't marked SUCCESSFUL), the A/B stack falls back to the other slot, which still has the previous version of Android in it."

Because the *previous* slot was never touched by the update (A/B writes only the inactive slot) and is still marked successful, fallback is to a fully-intact OS. This is the mechanism that turns "a bad update" into "an automatic, silent revert" instead of a brick. It does **not** depend on the Helix agent running, the network being up, or any server signal — which is precisely why it is trustworthy.

---

8. Rollback Index / Anti-Downgrade

Source: AVB 2.0 README, Boot flow, AVB.

Rollback *protection* (anti-downgrade) is distinct from the automatic *rollback* of §7. The §7 rollback reverts to the **immediately-previous** good slot for reliability. Anti-downgrade prevents an attacker (or a malicious/old package) from forcing the device onto an **older, known-vulnerable** firmware even via an otherwise-valid update path.

Mechanism:

- Each signed image (vbmeta) carries a **`rollback_index`** value at a specific **rollback index location** (AVB supports multiple locations, so different partitions / chained vbmeta can have independent indexes).
- The device persists a **`stored_rollback_index`** per location in **tamper-evident storage** (RPMB / dedicated persistent partition / TEE — **board-dependent, UNVERIFIED** for the target).
- At boot, the bootloader permits the image only if **`image.rollback_index >= stored_rollback_index`** for each location. An older image (lower index) is rejected.
- The `avb_ops` callbacks **`read_rollback_index()`** and **`write_rollback_index()`** abstract the storage backend.

The load-bearing ordering rule. The AOSP docs are explicit and this is the single most important operational constraint: for A/B devices, **the slot must be marked `SUCCESSFUL` *before* the stored rollback index is updated.** Quoting the boot-flow guidance: the boot slot must be marked `SUCCESSFUL` via the Boot Control HAL **before rollback protection metadata is updated** — and the OTA implement page reinforces that marking the slot `slot-successful` must occur before updating rollback-protection metadata "to prevent boot failures if the device loses power during the update process."

Why it matters: if the stored rollback index were bumped *before* the new slot proved itself good, and the new slot then failed, the bootloader's automatic fallback (§7) would try the old slot — but the old slot now has a `rollback_index` **lower than** the freshly-bumped `stored_rollback_index`, so anti-downgrade would **reject it**, leaving the device with no bootable slot. Bumping the stored index only after a confirmed-good boot keeps fallback always-possible.

---

9. How the Stack Delivers the Zero-Corruption Guarantee

The "zero-corruption" property is not one feature; it is the *composition* of independent, on-device, signed mechanisms — each of which fails safe:

Layer	Guarantee it adds	Fail-safe behaviour
A/B slots (`update_engine`)	The running slot is never modified; the update goes only to the *inactive* slot.	A failed/partial write never touches the slot you're running.
AVB / vbmeta	The image to be booted is cryptographically authenticated to the OEM (or user) root key.	Tampered/unsigned image → not GREEN/YELLOW → blocked (locked device).
dm-verity (+ FEC)	Every read-only block matches the signed hash tree at runtime.	Mismatch → **EIO** (looks like corruption); FEC repairs isolated rot.
`update_verifier` + care map	The blocks the OTA *wrote* are force-verified on first boot **before** commit.	Corrupt download/write caught before the good slot is given up.

Layer	Guarantee it adds	Fail-safe behaviour
`boot_control` retry-count + fallback	A new slot must *prove* itself within N boots, else automatic revert.	Never-successful slot → automatic fallback to prior good slot.
Rollback index (anti-downgrade)	Device can only move to an image ≥ stored index; old vulnerable images rejected.	Ordering rule keeps fallback always possible despite the monotonic index.

The key architectural insight for Helix: **the entire dangerous part is on-device, signed, and bootloader-enforced.** None of it depends on a correct server, a live network, or a running Helix agent at the critical moment. The server's only way to *break* this guarantee is to ship a payload that is mis-signed or mis-built — which is a *build-pipeline* concern, not an apply-time concern.

---

10. What the Helix Agent MUST and MUST NOT Do

This is the operative section for Helix implementation. The agent is a *client of* the stack above; it must never become a *substitute for* any part of it.

MUST

- **Deliver a correctly-signed, correctly-built payload.** AVB/vbmeta signing and dm-verity hashtree generation are produced by the **build pipeline** (`avbtool`, the build's AVB integration, `ota_from_target_files`). The package the agent installs must already be valid for the device's locked root key. (See `aosp-update-engine.md` for payload generation; signing-flag specifics are UNVERIFIED there and should be pinned.)
- **Drive the update only through `update_engine`** (`applyPayload` / the AIDL surface). `update_engine` is what legitimately calls `setActiveBootSlot` and the rest of the `boot_control` HAL in the correct order.
- **Let the framework own `markBootSuccessful`.** The agent may *report* that the system reached a healthy state, but the actual commit is `update_verifier` → framework → HAL. If Helix adds an application-level health gate, it should *gate its own "report healthy" signal*, not bypass the HAL sequence.
- **Observe, for telemetry only:** `ro.boot.verifiedbootstate`, `ro.boot.veritymode`, current/active slot, `slot-successful`, and Virtual A/B merge state. Detecting an unexpected slot fallback or a verity EIO event is exactly the high-value signal the Go control plane wants for recall/rollout decisions.
- **Model the trial window.** Treat "applied / pending-reboot / booted-not-yet-successful / successful / rolled-back" as first-class states in telemetry, mirroring the boot_control state machine.

MUST NOT

- **MUST NOT** directly write slot flags (`active`/`bootable`/`successful`) or reset/alter `slot-retry-count` outside `update_engine` / documented HAL usage.

- **MUST NOT** call `markBootSuccessful` itself to "force" a commit. That defeats `update_verifier` and the trial-boot safety window.
- **MUST NOT** write or bump the **rollback index** / `stored_rollback_index`, and MUST NOT reorder the "mark SUCCESSFUL before bump index" sequence. This is the one mistake that can leave a device with no bootable slot.
- **MUST NOT** regenerate, strip, or re-sign **vbmeta** on-device, disable AVB, or boot with verification off (`vbmeta` flags / `--disable-verity`). On a production fleet the device should stay GREEN/YELLOW.
- **MUST NOT** disable dm-verity or mount protected partitions read-write to "patch in place." All updates go through the A/B payload path; in-place mutation breaks the hashtree and triggers EIO.
- **MUST NOT** assume the agent must be alive for rollback to work. Rollback is bootloader-enforced; the agent must not insert itself as a single point of failure in the safety path.

---

11. Implications for Helix OTA

- **The safety guarantee is inherited, not built.** Helix gets zero-corruption + automatic rollback "for free" *provided* the device is a properly-configured A/B + locked-AVB Android 15 device and the payload is correctly signed. Helix's engineering risk is concentrated in the **build/sign pipeline** and in **not interfering** with the on-device stack — not in re-implementing apply/verify/rollback.
- **Control-plane value-add is orthogonal to safety.** Staged rollout %, cohorts, recall, inventory, and telemetry all sit *above* this stack. The Go server should consume slot/verity/merge telemetry to make rollout decisions, but it has **no role in the per-device safety enforcement**.
- **Anti-downgrade interacts with rollout policy.** Because the stored rollback index is monotonic, a Helix "recall to previous build" is only possible if the previous build's `rollback_index` is still ≥ the device's stored index. The control plane must understand that **bumping rollback_index in a release is a one-way door** for that device — plan recalls accordingly, and avoid unnecessarily incrementing `rollback_index` between closely-spaced builds.
- **Telemetry must capture the failure signatures.** Unexpected slot fallback (current slot != the slot Helix told it to boot), `verifiedbootstate` != GREEN/YELLOW, and verity EIO are the concrete signals that an update failed safe. These should be reported and should drive automatic rollout halt.
- **Board reality check is required.** All of the above presumes the RK3588 / Orange Pi 5 Max target actually ships a conformant `boot_control` HAL, locked AVB, and a real rollback-index storage backend. This is **UNVERIFIED** and must be confirmed on hardware before the guarantee can be claimed for the Helix fleet (see continuation).

---

12. Open Questions / UNVERIFIED Items

1. **Default `slot-retry-count`** — commonly cited as 3 (sometimes 7); exact default is board/bootloader-dependent. UNVERIFIED.

2. **Android 15 `android.hardware.boot` AIDL (`IBootControl`)** — exact method set and the precise `update_engine`?HAL call order were not byte-confirmed against the Android 15 tag. UNVERIFIED.
3. **Rollback-index storage backend on the target** — RPMB vs dedicated persistent partition vs TEE-backed; determines tamper-resistance. UNVERIFIED for RK3588/Orange Pi 5 Max.
4. **Target board AVB lock state & signing** — whether the Orange Pi 5 Max / RK3588 build ships locked AVB (GREEN/YELLOW) with proper vbmeta signing, or runs UNLOCKED (ORANGE), in which case the guarantee is weaker. UNVERIFIED — needs hands-on confirmation.
5. **Exact `update_verifier` → `markBootSuccessful`/`setBootSuccessful` call site** in current `bootable/recovery` (the function is sometimes wrapped via `setSlotAsBootable`/framework `BootControl`); trace against the current tree. UNVERIFIED spelling.
6. **EIO vs enforcing verity mode default** on the target and its effect on first-boot failure handling. UNVERIFIED for the board.
7. **Persistent digest / chained-vbmeta specifics** for partitions whose size/root-hash is not known at vbmeta-build time — relevant only if Helix updates such partitions. Not deeply explored here.

---

13. Sources Consulted

Primary (AOSP, official): - Verified Boot (overview) — https://source.android.com/docs/security/features/verifiedboot - Boot flow (verifiedbootstate GREEN/YELLOW/ORANGE/RED; mark SUCCESSFUL before rollback metadata) — https://source.android.com/docs/security/features/verifiedboot/boot-flow - Android Verified Boot (AVB) — https://source.android.com/docs/security/features/verifiedboot/avb - Android Verified Boot 2.0 README (rollback index, chain partitions, `read_rollback_index`/`write_rollback_index`, `avb_ops`) — https://android.googlesource.com/platform/external/avb/+/master/README.md - Implement dm-verity (hash tree, 0xb001b001 metadata, RSA-2048, FEC, EIO on failure) — https://source.android.com/docs/security/features/verifiedboot/dm-verity - Implement OTA updates / bootloader (boot_control HAL: `setActiveBootSlot`, `markBootSuccessful`, `getCurrentSlot`, `isSlotBootable`, `setSlotAsUnbootable`, slot-retry-count, fallback, ordering rule) — https://source.android.com/docs/core/architecture/bootloader/updating - `update_verifier.cpp` (care map verification, ro.boot.veritymode EIO vs enforcing, markBootSuccessful) — https://android.googlesource.com/platform/bootable/recovery/+/master/update_verifier/update_verifier.cpp - HAL header `hardware/libhardware/include/hardware/boot_control.h` (referenced by the OTA-implement page as the HAL contract).

Companion Helix note: - `aosp-update-engine.md` (this repo) — A/B + Virtual A/B mechanics, payload generation, `applyPayload` client API, `update_verifier` triggering dm-verity before zygote.

No statistics, dates, stars, or citations were fabricated. Items not directly verified against the current docs revision or against the target board are tagged "UNVERIFIED — needs confirmation."

---

14. Confidence

Overall confidence: HIGH on the architecture and the must/must-not guidance — the AVB chain of trust, dm-verity behaviour, the `boot_control` slot state machine, `update_verifier`'s care-map verification, automatic retry-count fallback, and the rollback-index ordering rule are all stable, well-documented AOSP mechanisms confirmed across multiple primary sources. **MEDIUM** on exact constants (default retry count), the precise Android 15 `IBootControl` AIDL surface, and the exact `update_verifier` commit call site — flagged UNVERIFIED. **LOW/ UNVERIFIED** on target-board specifics (RK3588 / Orange Pi 5 Max AVB lock state, vbmeta signing, rollback-index storage backend), which must be confirmed on hardware before the zero-corruption guarantee can be asserted for the Helix fleet.